

Student Marks Manager

Instructor Review Abstract

Project Type Full-stack web application	Primary Goal Manage student marks and generate results	Core Stack React + Node.js + Java + Neon PostgreSQL
---	--	--

Abstract

The **Student Marks Manager** is a simple full-stack web application designed to help a teacher or administrator store, edit, and review student academic records. The application allows users to enter a student's roll number, name, subject marks, and attendance, then automatically calculates total marks, percentage, grade, and result status. React is used to create the responsive user interface, Node.js with Express is used as the REST API and application server, Java is used as the calculation engine for academic result logic, and Neon PostgreSQL is used for persistent cloud storage. The design intentionally keeps the project small: it uses one main database table, a small set of REST endpoints, and a clear separation between presentation, API/database work, and Java calculation logic. The completed system can be packaged as one Docker service for simple deployment and can be developed and demonstrated without paid infrastructure.

Problem Statement

Manual calculation and maintenance of student marks can lead to arithmetic errors, repeated work, and difficulty finding records. A lightweight web-based system can centralize student information and automatically generate consistent academic results.

Objectives

- Provide a simple interface to add, view, edit, and delete student records.
- Store student data persistently in a PostgreSQL database hosted by Neon.
- Use Java to calculate total marks, percentage, grade, pass/fail status, and attendance warning status.
- Expose the application through REST APIs using Node.js and Express.
- Create a project that is small enough for beginners to understand, test, and deploy.

Expected Result

A working web application where the user can create a student record, immediately receive a calculated result, and later search, view, update, or remove the record. The application should demonstrate the practical use of three programming technologies in a single system without unnecessary complexity.

Recommended presentation: Show this first page to the instructor for project approval. The following pages contain the implementation specification to use after approval.

1. Scope and System Architecture

Keep Version 1 deliberately small. Do not add authentication, payments, file uploads, email, notifications, or role management unless the instructor later asks for them.

Layer	Technology	Responsibility
Frontend	React + Vite	Forms, student table, result display, basic validation
Backend	Node.js + Express	REST API, validation, Neon connection, Java process control
Calculation	Java	Total, percentage, grade, pass/fail, attendance status
Database	Neon PostgreSQL	Persistent student records
Packaging	Docker	Run Node and Java together; serve built React app

User Features - Version 1

- Dashboard: show student count and basic class averages.
- Students page: list all students with percentage, grade, and attendance status.
- Add/Edit form: name, roll number, Maths, Java, DBMS, and attendance percentage.
- Actions: view result, edit record, delete record.
- Search by student name or roll number.

Data Flow

React form -> POST /api/students -> Node validates -> Java calculates -> Node writes to Neon -> JSON response -> React updates table.

Recommended Project Structure

```
student-marks-manager/  
- client/ # React + Vite  
  - src/  
    - components/  
    - pages/  
    - App.jsx  
    - package.json  
- server/ # Node + Express  
  - src/  
    - db.js  
    - javaRunner.js  
    - server.js  
    - package.json  
- java/  
  - StudentCalculator.java  
- sql/  
  - schema.sql  
- Dockerfile  
- .env.example  
- .gitignore  
- README.md
```

2. Exact Implementation Contract

The following requirements define the Version 1 implementation. Keep the architecture unchanged unless a practical implementation issue requires a small adjustment.

Database Schema

```
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,  
  roll_no VARCHAR(20) UNIQUE NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  maths INT NOT NULL CHECK (maths BETWEEN 0 AND 100),  
  java INT NOT NULL CHECK (java BETWEEN 0 AND 100),  
  dbms INT NOT NULL CHECK (dbms BETWEEN 0 AND 100),  
  attendance NUMERIC(5,2) NOT NULL CHECK (attendance BETWEEN 0 AND 100),  
  total INT NOT NULL,  
  percentage NUMERIC(5,2) NOT NULL,  
  grade VARCHAR(2) NOT NULL,  
  result VARCHAR(10) NOT NULL,  
  attendance_status VARCHAR(20) NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

REST API

Method	Route	Purpose
GET	/api/students	Return all students
GET	/api/students/:id	Return one student
POST	/api/students	Validate, calculate, insert
PUT	/api/students/:id	Validate, recalculate, update
DELETE	/api/students/:id	Delete student
GET	/api/health	Return service health

Java Calculation Rules

- Input: maths, java, dbms, attendance.
- $\text{total} = \text{maths} + \text{java} + \text{dbms}$.
- $\text{percentage} = \text{total} / 3.0$.
- Grade: A for ≥ 90 , B for ≥ 80 , C for ≥ 70 , D for ≥ 60 , F otherwise.
- Result: PASS when every subject is ≥ 40 and percentage ≥ 40 ; otherwise FAIL.
- Attendance status: OK when attendance ≥ 75 ; otherwise WARNING.
- Java program should accept one JSON object from stdin and return one JSON object to stdout. Use a simple JSON library such as Jackson; do not build an HTTP Java server.

Required Java Output Shape

```
{"total":249,"percentage":83.00,"grade":"B","result":"PASS","attendance_status":"OK"}
```

Important: Node.js is the only public backend. React never connects directly to Neon. Java is an internal calculation process started by Node. This keeps the architecture understandable and makes one-container deployment easy.

3. Build Order and Deployment Instructions

1. Install tools

Install Node.js LTS, a JDK, Git, VS Code or another IDE, and Docker Desktop. Create a GitHub account if needed.

2. Create project

Create the repository and the client, server, java, and sql directories exactly as specified.

3. Create Neon database

Create a Neon PostgreSQL project, copy its connection string, and store it only in a local .env file as DATABASE_URL.

4. Build Java first

Write StudentCalculator.java, compile it into a JAR, test sample inputs, and confirm the JSON output is correct.

5. Build Node API

Create the Express server, PostgreSQL connection, validation, CRUD routes, and javaRunner module. Test every endpoint with curl or Postman.

6. Build React UI

Create the form, table, dashboard, edit/delete actions, loading states, and error messages. The frontend should call only the Node API.

7. Connect everything

Test add, edit, delete, search, calculation, and page refresh persistence. Confirm data remains in Neon after restarting the server.

8. Dockerize

Build one image containing Node.js, the compiled Java JAR, and the React production build. Node serves the React build and the /api routes.

9. GitHub

Commit the working project, but never commit .env or secrets. Add README instructions and screenshots.

10. Deploy

Deploy the Docker web service on a free-tier service such as Render and set DATABASE_URL as an environment variable. Neon remains the database.

Environment Variables

```
DATABASE_URL=your-neon-postgres-connection-string
PORT=3000
NODE_ENV=production
```

Minimum Test Cases

Input	Expected
80, 90, 70, 88	Total 240, Percentage 80.00, Grade B, PASS, OK
39, 80, 90, 90	Percentage 69.67, Grade D, FAIL because one subject is below 40
90, 90, 90, 74	Grade A, PASS, Attendance WARNING
Negative or >100 mark	Reject request with validation error; do not write to DB
Duplicate roll number	Reject with clear conflict message

4. Deliverables and Implementation Handoff

Final Deliverables

- Source code on GitHub.
- Working React interface.
- Working Node/Express REST API.
- Java calculation module/JAR.
- Neon PostgreSQL database schema.
- Dockerfile and README.
- Live deployment URL and screenshots for the final report/demo.

Implementation Handoff Specification

Build the **Student Marks Manager** exactly from the requirements in this document. Use React with Vite for the frontend, Node.js with Express as the only public backend, Java as an internal command-line calculation engine launched by Node, and Neon PostgreSQL for persistent data. Keep Version 1 limited to CRUD, search, result calculation, dashboard statistics, and validation. Do not introduce authentication, microservices, Redis, GraphQL, WebSockets, or extra frameworks unless required by a clear implementation issue. Create the exact folder structure, database schema, REST routes, Java JSON input/output contract, React pages/components, Dockerfile, .env.example, README, seed data, and tests. The final application must run locally, persist records in Neon, and deploy as one Docker web service. Explain each file, command, and integration point clearly, and verify each stage before proceeding.

Suggested User Interface

Dashboard: student count, average percentage, average attendance.

Students: searchable table with Roll No, Name, Percentage, Grade, Attendance, Actions.

Add/Edit: Name, Roll Number, Maths, Java, DBMS, Attendance, Save button.

Result: Total, Percentage, Grade, PASS/FAIL, Attendance status.

Deployment Notes

Use GitHub for source control, Neon PostgreSQL Free for the database, and a free-tier Docker-capable web service such as Render for the backend/container. Keep secrets in environment variables. Free-tier quotas and policies can change. so confirm the current provider limits at deployment time.

Project principle: keep the first release small, stable, and easy to demonstrate. Add advanced features only after CRUD, calculation, persistence, and deployment are working end-to-end.